Programming 1

Laszlo SZATHMARY
University of Debrecen
Faculty of Informatics

Lecture #8

- modules
- random numbers

(last update: 2026-01-17 [yyyy-mm-dd])

2025-2026, 2nd semester

Modules

As our program gets longer, the need arises to cut it into several source files.

First, maintenance would be easier. The project would be clearer, simpler.

Second, we might want to reuse some useful functions in different programs, without copying the functions to every program.

Exercise:

Write two programs:

1. Print prime numbers that are less than 100 (`ex1.py`)
2. Print the sum of prime numbers that are less than 200 (`ex2.py`)

Link: https://arato.inf.unideb.hu/szathmary.laszlo/pmwiki/index.php?n=EnPy3.20121110a

ex1.py

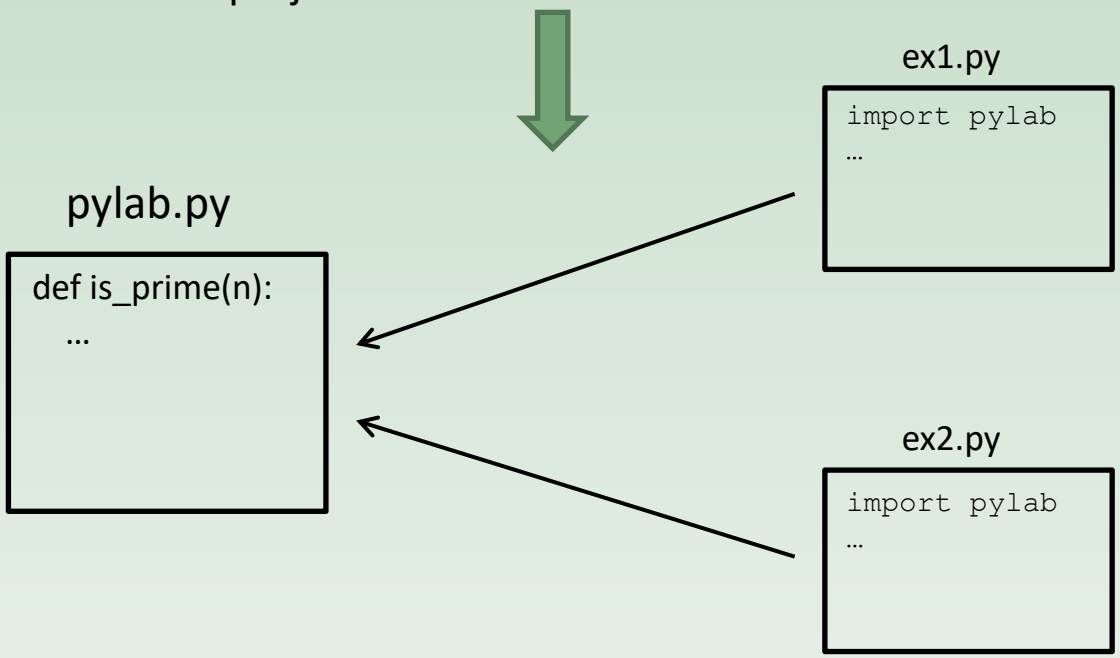
```
def is_prime(n):
    …
```

ex2.py

```
def is_prime(n):
    …
```

When ready, let's investigate what is common in the two scripts. The common part is the `is_prime()` function.

1. the very same function is repeated at different places
2. the `is_prime()` function performs an operation that could be useful in the future in another project



Move the common function to a module (e.g. `pylab.py`), and import this module in the scripts.

pylab.py

```python
def is_prime(n):
    """
    Decide whether a number is prime or not.
    """
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    i = 3
    maxi = n**0.5 + 1
    while i <= maxi:
        if n % i == 0:
            return False
        i += 2

    return True


def hello():
    return "Hello, World!"
```

Usage (ex3.py):

```python
import pylab


def main():
    print(pylab.is_prime(5))

    print(pylab.hello())
```

`import pylab` # without '.py'

Variations

```
import pylab
```

It won't put the functions defined in `pylab` to the current symbol table. Only the name of the module (`pylab`) is put in the symbol table.
If you want to use the module's functions, then refer to them using the module's name:
`pylab.is_prime(n)`

```
import  pylab as pl
```

If the module has a long name and/or we want to refer to it several times, then you can rename it by putting an alias on it. Then:
`pl.is_prime(n)`

Variations (cont.)

```
from pylab import is_prime
```

Meaning: from the `pylab` module we import just the function name `is_prime` to the symbol table.

!!! It doesn't bring in the module's name to the symbol table **!!!**

Ex.: `print(pylab.hello())` # error, the symbol „pylab" is unknown

Solution:

1. `from pylab import is_prime, hello`

2. `from pylab import is_prime`
   `import pylab`

3. `from pylab import *`
   Not recommended, since we won't know what is imported from where. It just causes a mass.

Testing a module

Before:

```python
def is_prime(n):
    """
    Decide whether a number
    is prime or not.
    """
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    i = 3
    maxi = n**0.5 + 1
    while i <= maxi:
        if n % i == 0:
            return False
        i += 2

    return True


def hello():
    """
    Classic example.
    """
    return "Hello, World!"
```

After:

```python
#!/usr/bin/env python3

"""
pylab module
============

This module was made for the Python lab.
Functions used often are collected here.
"""

def is_prime(n):
    """
    Decide whether a number is prime or not.
    """
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    i = 3
    maxi = n**0.5 + 1
    while i <= maxi:
        if n % i == 0:
            return False
        i += 2

    return True


def hello():
    """
    Classic example.
    """
    return "Hello, World!"

##############################

if __name__ == "__main__":
    for n in range(2, 20):
        if is_prime(n):
            print("{n} is prime".format(n=n))
```

docstring

test

Testing a module (cont.)

```
37
38  ##############################
39
40  if __name__ == "__main__":
41      for n in range(2, 20):
42          if is_prime(n):
43              print("{n} is prime".format(n=n))
```

arbitrary tests

The condition (line 40) is <u>true</u>, if the module is <u>executed directly</u> from the command-line (i.e. `./pylab.py`). In this case the test will run.

If the module is <u>imported</u>, then the condition is <u>false</u>, thus the test won't run.

That is: if we write a module, then (1) it can be used as a module, and (2) it can also be used as a stand-alone script!

Exercise: Modify the `pylab.py` module the followinf way. If it's executed directly, then it should read a number from the standard input, and then it should print whether this number is a prime or not.
Then execute the `ex1.py` script, in which import this modified `pylab.py` module. What do you notice?

Testing a module (cont.)

pylab.py

```python
#!/usr/bin/env python3


def hello():
    return "Hello, World!"

##########################

if __name__ == "__main__":
    print(__name__)
```

ex1.py

```python
#!/usr/bin/env python3

import pylab


def main():
    print(pylab.hello())
    print(pylab.__name__)

##########################

if __name__ == "__main__":
    main()
```

```
$ ./pylab.py
__main__
$
$ ./ex1.py
Hello, World!
pylab
```

Testing a module (cont.)

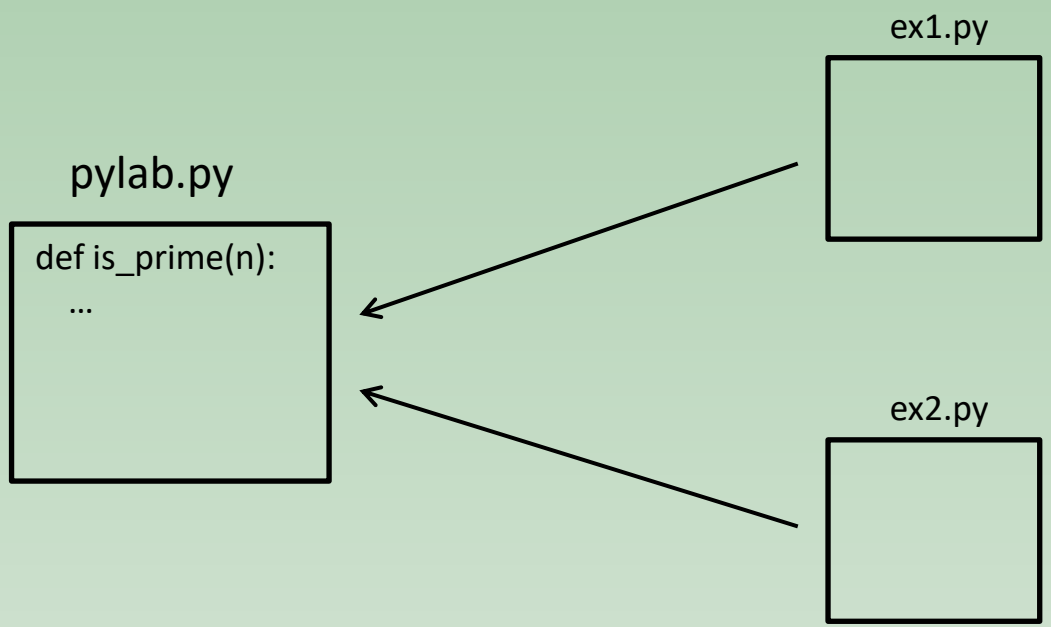
```
37
38  ##############################
39
40  if __name__ == "__main__":
41      for n in range(2, 20):
42          if is_prime(n):
43              print("{n} is prime".format(n=n))
```

Another advantage:

If you add tests to the end of your module, then you provide some concrete examples about the usage of your module. If somebody wants to use your module, (s)he will get some examples in the form of your tests.

„Alice: - I wrote a cool module that I just sent to you in email. Did you get it?
Bob: - Yeah, but… How can I use these functions?
Alice: - Go down to the bottom. There you will see some examples.
Bob: - Oh, OK. Cool! Thanks!"

Modules



Another advantage of using modules

If you modify a function, then you need to do it at one place only.

Exercise: In the `pylab.py` module replace the implementation of the `is_prime()` function to the much more efficient Miller-Rabin test.
You will find the source code of the MR test in the course material.

Then run the scripts `ex1.py` and `ex2.py` . What do you notice?

How a module is imported

```
import spam
```

We want to import this module. How will the interpreter find it? From where will it be imported?

1) The unterpreter checks if it's a built-in module.
   Built-in module: compiled in the interpreter.

```
>>> import sys
>>> sys.builtin_module_names
```

2) If it's not found, then it'll look for a file called `spam.py` in a list of directories. This list of directories can be found in `sys.path` .

```
>>> import sys
>>> sys.path
```

   This `sys.path` is initialized the following way:
   - the folder containing the script
   - The `PYTHONPATH` environment variable. It is similar to `PATH` : a colon-separated list of directories.
   - system folders specific to your Python installation

How a module is imported (cont.)

```
>>> import sys
>>> sys.path
```

`sys.path` is a normal list => after the initialization, you can also modify it in your script

!!! The folder containing the current script will be at the *beginning* of `sys.path` , i.e. it comes *before* the folders of the standard library. It can cause some strange errors if the folder contains some files whose names are identical to the names of some modules in the standard library !!!

Random numbers

https://docs.python.org/3/library/random.html

```
>>> import random
>>>
>>> random.random()
0.12214691572646652
>>> random.randint(1, 10)
9
>>> li = [3, 8, 2, 8, 4, 2, 1, 9]
>>> li
[3, 8, 2, 8, 4, 2, 1, 9]
>>> random.shuffle(li)
>>> li
[9, 8, 1, 4, 8, 2, 3, 2]
>>>
>>> random.choice(li)
8
```

`[0.0, 1.0)`

`lower <= N <= upper`

mixes (shuffles) the elements *in place* (random permutation)

selecting a random element from the list

Exercise: the `shuffle` method modifies the list *in place*. Write a function called `shuffled` that returns the shuffled list, allowing us to do the following for instance: `n = shuffled(li)[-1]`

homework

Exercises

1. [20121110a] modules
2. [20120905c] shuffled
3. Implement the **queue** and **stack** data structures with classes. Document everything with docstrings: the module, the class, and the class' functions.